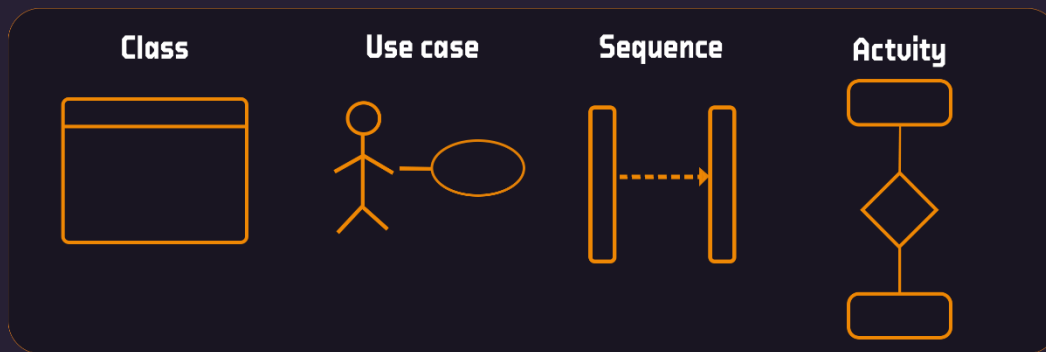


UML (UNIFIED MODELING LANGUAGE) DIAGRAMS



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Análisis y Diseño de Software	Unidad 1	Tema: Conceptos y diagramas del lenguaje unificado de modelado (UML)
	Semana #4	Instructora: Katya Michelle Asencio Bernal

Introducción:

Bienvenido al cuarto material de lectura de la Unidad #1 de Análisis y Diseño de Software. Luego de abordar las diferentes metodologías de desarrollo de software en el material anterior, en este material nos adentraremos en el estudio del Lenguaje Unificado de Modelado (UML). Desde sus orígenes históricos hasta la clasificación de sus 14 diagramas y sus elementos base, este bloque está diseñado para que domines la gramática visual indispensable en la industria actual.

Este material de lectura busca brindarte una base sólida en campo del Análisis y Diseño de Software, preparándote para enfrentar los desafíos y oportunidades que encontrarás en tu carrera profesional. A medida que avances en este material, te alentamos a participar activamente de las actividades, para favorecer tu proceso de aprendizaje.

Antes de comenzar, asegúrate de tener papel y lápiz a mano para las actividades interactivas.

Análisis y Diseño de Software

Sección 1:

1.4 Conceptos y diagramas del lenguaje unificado de modelado (UML)

Comencemos con el estudio del **Lenguaje Unificado de Modelado**, en esta sección abordaremos los pilares conceptuales y la naturaleza de los diagramas de UML.

Según Pressman (2010), el Lenguaje de Modelado Unificado (UML) es "un lenguaje estándar para escribir diseños de software. El UML puede usarse para visualizar, especificar, construir y documentar los artefactos de un sistema de software intensivo". En otras palabras, tal como los arquitectos de edificios crean planos para que los use una compañía constructora, los arquitectos de software crean diagramas de UML para ayudar a los desarrolladores de software a construir el software.

El UML se manifiesta a través de diagramas. Un diagrama es la representación gráfica de un conjunto de elementos, visualizados como un grafo de nodos (entidades) y arcos (relaciones).

Se utilizan diagramas porque el ser humano procesa de una manera más eficiente la información visual que los bloques densos de texto o código fuente. Estos diagramas sirven como la hoja de ruta que permite a los desarrolladores comprender la estructura y el comportamiento del sistema antes de escribir una sola línea de código.

Origen de UML

Grady Booch, Jim Rumbaugh e Ivar Jacobson desarrollaron el UML a mediados de los años noventa del siglo pasado con mucha realimentación de la comunidad de desarrollo de software. El UML fusionó algunas notaciones de modelado que competían entre sí y que se usaban en la industria del software en la época.

- En 1997 se envía la revisión UML 1.0 al Object Management Group (OMG), un consorcio sin fines de lucro involucrado en especificaciones de mantenimiento para su empleo en la industria de la computación. Tras varias revisiones, se adoptó el estándar UML 1.1.
- En 2005 se alcanzó el estándar UML 2.0, el cual representó una reestructuración profunda para soportar sistemas de mayor escala y complejidad.
- Actualmente el estándar vigente es UML 2.5.1, el cual está ratificado tanto por la OMG como por la ISO.

[illegible]

- **Independencia del Lenguaje y Proceso:** UML no es un lenguaje de programación ni define un proceso estándar (como el proceso en cascada). Está diseñado para ser compatible con la mayoría de los lenguajes de programación y procesos de desarrollo orientados a objetos existentes.
- **Capacidad de Extensión (Perfiles):** El estándar reconoce que ninguna herramienta puede cubrir todas las necesidades. Por ello, incluye mecanismos de extensión como los estereotipos, valores etiquetados y restricciones, permitiendo crear "Perfiles" específicos para distintas tecnologías sin alterar el lenguaje básico.

1.4.2 Diagramas que componen el UML.

Para representar un sistema de software de manera integral, el UML organiza sus herramientas en dos grandes familias de diagramas. Cada una se enfoca en una dimensión distinta del sistema: qué es (estructura) y cómo actúa (comportamiento).

Diagramas de Estructura

Estos diagramas muestran la estructura estática del sistema y sus partes en diferentes niveles de abstracción e implementación, así como la relación entre ellas. Los elementos en estos diagramas representan los conceptos significativos del sistema, pudiendo incluir conceptos abstractos, del mundo real o detalles técnicos de implementación.

Existen siete tipos de diagrama de estructura:

1. **Diagrama de clases:** Es el pilar del modelado orientado a objetos. Describe la estructura del sistema mostrando sus clases, atributos, operaciones y las relaciones entre objetos.
2. **Diagrama de componentes:** Describe cómo se divide el sistema en piezas físicas (módulos, librerías) y las dependencias entre ellas.
3. **Diagrama de despliegue:** Representa la arquitectura física, mostrando la configuración de los nodos de procesamiento y el software que reside en cada uno.
4. **Diagrama de objetos:** Muestra una instantánea de los elementos del sistema y sus relaciones en un momento específico del tiempo.
5. **Diagrama de paquetes:** Se utiliza para agrupar elementos del modelo en unidades lógicas, facilitando la organización de sistemas grandes.
6. **Diagrama de estructura compuesta:** Detalla la estructura interna de un clasificador (como una clase o componente), incluyendo sus partes y puertos.
7. **Diagrama de perfil:** Permite extender y adaptar el metamodelo de UML para plataformas o dominios específicos.

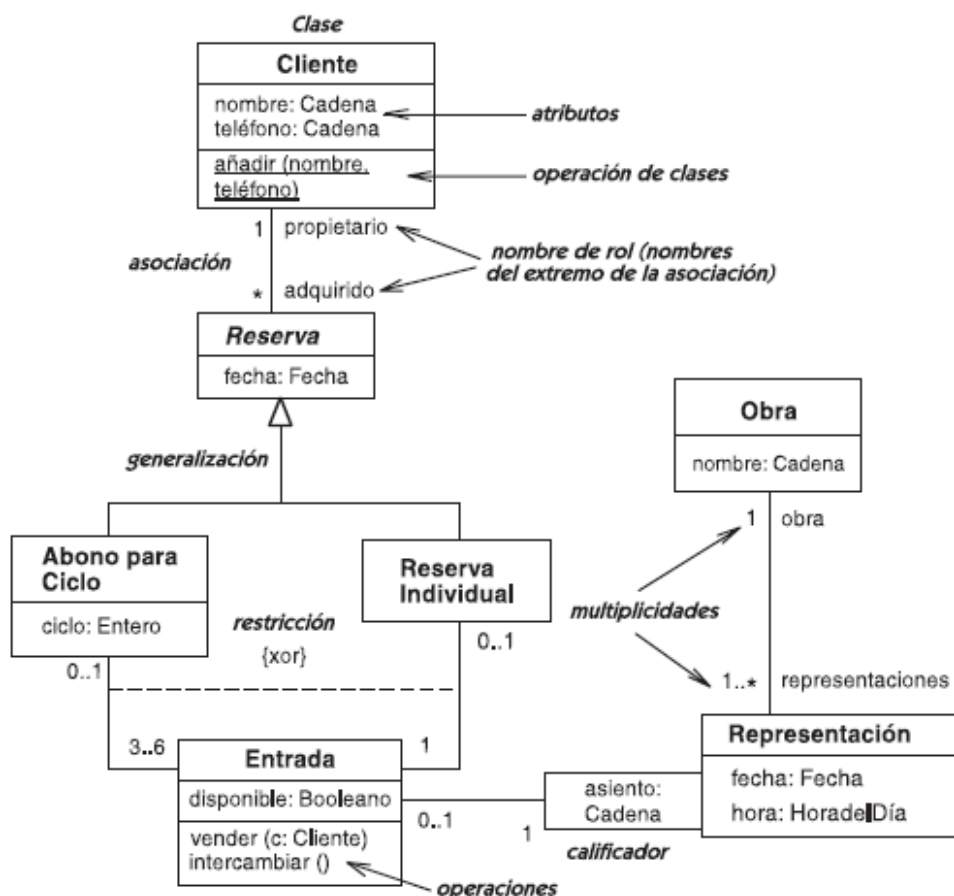


Diagrama de clases (Booch, 2007, p.28)

Diagramas de Comportamiento

Los diagramas de comportamiento muestran el comportamiento dinámico de los objetos en un sistema, que puede describirse como una serie de cambios en el sistema a lo largo del tiempo.

Existen siete tipos de diagramas de comportamiento:

1. **Diagrama de casos de uso:** Describe la funcionalidad del sistema a través de las interacciones entre los "actores" (usuarios o sistemas externos) y el sistema mismo.
2. **Diagrama de actividad:** Representa los flujos de trabajo paso a paso, permitiendo modelar la lógica de un negocio o un algoritmo complejo.

3. **Diagrama de máquina de estados:** Muestra los diferentes estados por los que pasa un objeto durante su ciclo de vida en respuesta a eventos.
4. **Diagrama de secuencia:** Se enfoca en el intercambio de mensajes entre objetos en una secuencia temporal específica para cumplir una función.
5. **Diagrama de comunicación:** Similar al de secuencia, pero pone el énfasis en la organización de los objetos y su relación estructural.
6. **Diagrama global de interacción:** Proporciona una visión de alto nivel al combinar diferentes diagramas de interacción en un solo flujo.
7. **Diagrama de tiempos (Timing):** Se utiliza para analizar el cambio de estado de los elementos en un eje de tiempo, muy común en sistemas de tiempo real.

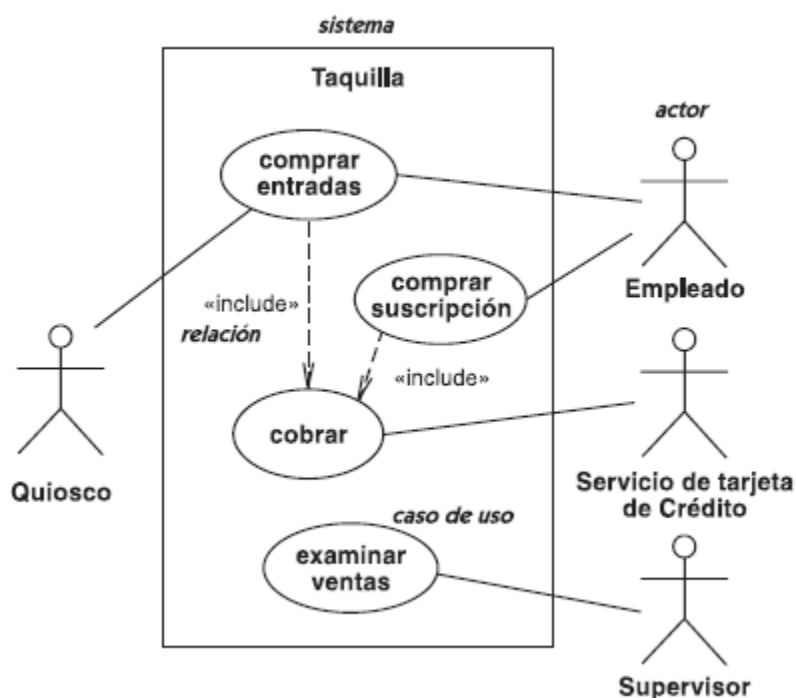


Diagrama de casos de uso (Booch, 2007, p.33)

Aunque existen 14 diagramas, en la práctica profesional no siempre se utilizan todos. Los más comunes en el Análisis y Diseño suelen ser los de Casos de Uso, Clases, Secuencia y Actividades.

1.4.3 Elementos de UML.

Para construir modelos coherentes, el UML se basa en tres bloques de construcción fundamentales que actúan como la gramática del lenguaje. De acuerdo con el Manual

de Referencia de UML, estos bloques son las Cosas, las Relaciones y los Mecanismos comunes.

Las "Cosas" (Things) en UML

Son las abstracciones que actúan como los componentes básicos del modelo. El manual las clasifica en cuatro grupos:

- **Cosas Estructurales:** Son los sustantivos de los modelos UML. Representan partes físicas o conceptuales del sistema.
 - *Ejemplos:* Clases, interfaces, colaboraciones, casos de uso, componentes y nodos.
- **Cosas de Comportamiento:** Son los verbos de los modelos. Representan el comportamiento a través del tiempo y el espacio.
 - *Ejemplos:* Interacciones (mensajes) y Máquinas de estado.
- **Cosas de Agrupamiento:** Son las partes organizativas de los modelos UML. Permiten dividir el sistema en piezas manejables.
 - *Ejemplo:* El Paquete (Package).
- **Cosas de Anotación:** Son las partes explicativas de los modelos. Permiten describir o comentar cualquier elemento.
 - *Ejemplo:* La Nota.

Relaciones (Relationships)

Las relaciones son el pegamento que une a las cosas, mostrando cómo se conectan y dependen entre sí. Las más importantes son:

1. **Dependencia:** Relación semántica donde un cambio en un elemento independiente afecta al elemento dependiente. Se representa con una línea discontinua con flecha.
2. **Asociación:** Relación estructural que describe un conjunto de enlaces entre objetos. Puede incluir la

Actividad

Interactiva #2



Lee las siguientes afirmaciones y marca si son Verdaderas (V) o Falsas (F) según lo aprendido:

El UML es un lenguaje de programación como Java o Python. []

El organismo encargado de mantener el estándar UML es la OMG. []

El UML es un lenguaje de "propósito general", lo que significa que se puede usar en muchos tipos de sistemas. []

El estándar actual y más reciente es el UML 1.1. []

Agregación (todo/parte débil) y la Composición (todo/parte fuerte).

3. **Generalización:** Relación de herencia donde el elemento especializado (hijo) comparte la estructura y comportamiento del general (padre).
4. **Realización:** Relación entre una especificación (interfaz) y su implementación (clase).

Mecanismos Comunes

Son los detalles que añaden precisión al lenguaje. El UML utiliza cuatro mecanismos constantes en todo el lenguaje:

- **Especificaciones:** Cada elemento tiene una descripción textual de su semántica.
- **Adornos:** Notaciones gráficas adicionales (como la visibilidad + o -).
- **Divisiones Comunes:** Por ejemplo, la distinción entre clase (abstracción) y objeto (instancia).
- **Mecanismos de Extensibilidad:** Estereotipos, valores etiquetados y restricciones.

1.4.4 Detalles específicos de UML.

Un modelo de software profesional no solo debe ser comprensible, sino técnicamente exacto. El UML proporciona detalles específicos que permiten a los desarrolladores comunicar restricciones de seguridad, reglas de negocio y adaptaciones tecnológicas de manera unívoca.

Visibilidad (Encapsulamiento)

De acuerdo con el Manual de Referencia UML, la visibilidad determina cómo un elemento puede ser visto y utilizado por otros clasificadores. Es la representación gráfica del principio de encapsulamiento de la programación orientada a objetos:

- **Público (+):** Cualquier elemento externo puede acceder a él.
- **Privado (-):** Solo el propio clasificador tiene acceso. Es vital para proteger el estado interno de los objetos.
- **Protegido (#):** El acceso se limita al propio clasificador y a sus descendientes (herencia).
- **Paquete (~):** Visible solo para elementos dentro del mismo paquete.

Multiplicidad y Roles en las Asociaciones

Para que un diseño sea útil en la implementación de bases de datos y lógica de negocio, se debe especificar la cantidad de objetos que participan en una relación:

- **Multiplidad:** Indica el rango de instancias que pueden conectarse (ejemplo: 1, 0..*, 1..n).
- **Roles:** Es un nombre colocado al final de una línea de asociación para indicar el papel que desempeña el objeto. Por ejemplo, en una relación entre Persona y Empresa, el rol de la persona podría ser "empleado".

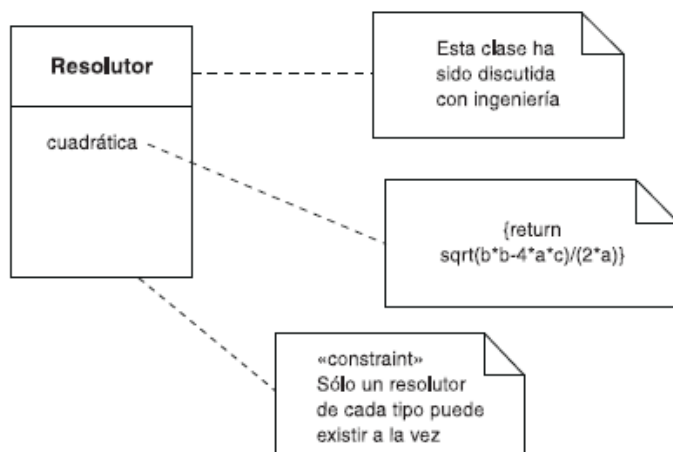
Mecanismos de Extensibilidad

El manual de referencia destaca que el UML es un lenguaje "abierto". Esto significa que, aunque tiene un núcleo fijo, permite añadir nuevos conceptos mediante tres mecanismos:

1. **Estereotipos:** Permiten crear nuevos tipos de bloques de construcción a partir de los existentes. Se escriben entre guillemets (<< >>).
 - *Ejemplos comunes:* <<interface>>, <<entity>>, <<control>>, <<boundary>>.
2. **Valores Etiquetados:** Son propiedades para añadir información nueva a la especificación de un elemento (como el autor de la clase o la fecha de creación). Se representan como {etiqueta = valor}.
3. **Restricciones:** Son reglas que modifican la semántica de un elemento. El UML incluye el **OCL (Object Constraint Language)** para escribir estas reglas de forma formal.
 - *Ejemplo:* {salario > 0} o {edad >= 18}.

Notas y Comentarios

Las Notas son elementos gráficos que contienen información textual o restricciones.



Son fundamentales para documentar el "porqué" de una decisión de diseño, permitiendo que la documentación técnica viva dentro del mismo modelo visual.

Notas (Booch, 2007, p.477)

Bibliografía

Pressman, R. S. (2010). *INGENIERIA DE SOFTWARE* (7th ed.). McGraw-Hill

Interamericana de España S.L.

Rumbaugh, J. (2007). *El lenguaje unificado de modelado* 2/e. Pearson Educación.

¿Qué es el Lenguaje Unificado de Modelado (UML)? (s.f.). Obtenido de Visual

Paradigm: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/#composite-structure-diagram>

